

Confidentiality Across Agent Handoffs

Conditions for preserving an inference guarantee through tools and delegation

Imran Siddique · OPAQUE Systems · September 27, 2026

Technical report, version 1 · Component and composed software experiments

Abstract

This paper states conditions under which tool calls and agent handoffs preserve an authorized plaintext-holder boundary. This requires a protected channel bound to an appraised workload, restricted authority, enforceable downstream information-flow rules, and control over every other plaintext sink. A signature on an execution record or a valid hardware quote alone cannot establish these conditions. If a recipient cannot satisfy them, the sender must withhold the data or obtain authorization for a precisely described disclosure. This paper develops a conditional composition argument, a proposed handoff contract, and component experiments that expose failures of appraisal, supervision, and outcome inference. A composed software harness joins provisioning, diagnostic model computation, a confined agent, a mediated tool, delegated peer authentication, and exact-output disclosure. Its paired mutations expose earlier leaks despite successful final delivery; a rerun on September 27, 2026 reproduced all 36 recorded observations. All results are software results with synthetic attestation and a single operator. AgenTrust supplies relevant identity, key-release, gateway, delegation, and evidence primitives, but its present components do not demonstrate the complete property. The distinction matters most at remote tools, CPU-GPU transfers, operator-controlled key brokers, runtime changes, and audit systems.

Scope of this edition. This is an author-prepared technical report, not peer reviewed and not a normative specification. Version 1 is a bounded software edition. Attestation in the composed experiment is synthetic, signed by test-owned roots. One operator runs every party, including the tool, the delegated peer and the observers. No model key or model computation is protected by hardware, and no GPU path is exercised. Hardware acceptance on an owner-controlled launch host and composition with independently operated peers are future work; section 8.5 records the status of each acceptance item as of September 27, 2026. Two cloud hardware diagnostics are summarized in section 7 (GCP on September 17 and 18, Azure on September 18). Their raw captures contain device-identifying data and are not published with this report. Component results preserve the counts recorded at their pinned revisions. The composed software experiment was rerun for this edition; the record is in the accompanying experiments/rerun-2026-09-27.txt.

Patent Pending.

1. The guarantee that must survive

A June 2025 design report by Pattern Labs (now Irregular), written with a frontier-model developer, distinguishes confidentiality of inference inputs and outputs from confidentiality of model weights. Its architecture considers attested execution, model provisioning, and the confidential boundary, including accelerator paths. That analysis motivates examining the whole data path. It does not establish that arbitrary external tools inherit an inference service's protections. [Confidential Inference Systems: Design principles and security risks, June 2025](#)

Consider an agent analyzing a private contract. It sends a paragraph to a translation tool, stores an embedding, then asks another agent to identify renewal obligations. The initial inference service may protect memory from its infrastructure operator. Each subsequent operation introduces another possible plaintext holder: a tool process, embedding provider, database, remote model, tracing service, or privileged administrator.

The relevant question is therefore asset-specific: **who may learn which information, through which execution paths, under which assumptions?** “Confidential” without an asset, adversary, recipient set, and leakage boundary is insufficient.

We distinguish three properties:

Property	Protected asset	Example failure
Operator confidentiality	Inputs, outputs, and intermediate state	A tool operator reads request logs.
Authorized information flow	Information derived from protected inputs	An approved agent sends a secret to an unauthorized recipient.
Model custody	Model weights and related secrets	An operator-controlled broker releases the decryption key to a substituted workload.

The first protects against infrastructure access. The second constrains software that legitimately sees plaintext. The third can remain intact even when a tool leaks user data. A deployment must report these properties separately.

2. Threat model and trust assumptions

The proposed design considers a malicious network, an infrastructure administrator outside the accepted hardware boundary, a substituted peer or tool, replayed evidence, stale policy, and malicious instructions in model inputs or tool results. Application behavior is untrusted until constrained by an approved implementation and its enforcement boundary. Model-generated assertions of permission have no authority.

The trusted computing base includes hardware and firmware accepted by deployment policy; evidence verifiers and trust roots; the measured workload and its dynamic-loading controls; channel and key handling; policy evaluation; release authorities; and all components able to read protected plaintext. A VM-level boundary includes its guest operating system and privileged guest software. Shared isolation between a tool and its policy engine enlarges that trusted base.

Cryptographic primitives are assumed sound. Accepted hardware must meet the stated adversary model. Physical extraction, speculative or ciphertext side channels, firmware compromise, and traffic analysis require explicit platform-specific treatment. A deployment cannot convert an excluded adversary into a covered adversary by adding an attestation check.

Availability and semantic correctness of model answers are outside the confidentiality claim. Complete elimination of timing, message-length, access-pattern, and termination leakage is also outside the proposed baseline. The permitted leakage must be documented. Deployments requiring stronger secrecy need separate analysis of padding, scheduling, storage access, and covert channels.

3. A conditional composition argument

Represent a workflow as a graph whose vertices are execution or storage domains and whose edges carry data. For each asset x , define an authorized recipient set $R(x)$, an allowed-purpose policy, a retention rule, a declassification authority, and an explicit leakage allowance. Derived data inherits the relevant restrictions unless an authorized transformation changes them.

Let $B(x)$ contain every approved domain permitted to hold plaintext derived from x . An individual vertex belongs in $B(x)$ only when its execution protections and application controls satisfy the policy for x . A vendor certificate

or endpoint name does not establish membership by itself.

For each transfer from domain A to domain B, require:

1. **Recipient appraisal.** The sender accepts B's evidence under locally trusted reference values, platform policy, freshness requirements, and revocation rules. The evidence binds the destination channel key to the accepted execution domain. A signed report containing an arbitrary public key is insufficient unless the accepted workload creates and retains the corresponding private key under protection.
2. **Authority.** The caller has authority for the specific asset, operation, recipient, purpose, and interval. Further delegation cannot increase that authority. Recipient identity and evidence identity must refer to the same party in the same transaction.
3. **Protected transfer.** Encryption terminates inside B's accepted domain. Relays, TLS terminators, error handlers, and queues outside it receive no protected plaintext. Request and response channels both satisfy this rule.
4. **Complete mediation.** Every plaintext exit from B is controlled: network traffic, tool arguments, files, model-provider requests, debugging, logs, crash dumps, checkpoints, and returned results. Internal transformations cannot silently strip restrictions.
5. **Continuity.** The workload and policy used for the operation remain those accepted during appraisal. Changes invalidate the authorization or trigger a new appraisal. Key lifetimes and session rules bound the interval of trust.

Conditional claim. Given an initially protected asset, these conditions preserve its authorized plaintext-holder boundary across any finite sequence of mediated transfers, excluding the declared leakage and authorized declassifications.

The argument is inductive. Initially, all plaintext holders are approved. At each transfer, mediation either blocks release or admits one appraised and authorized recipient over a protected channel. The receiver's controls constrain subsequent copies and derivatives. The approved-holder property therefore remains true after that step. Composition fails as soon as an unmediated sink, unauthorized recipient, false appraisal, or unenforced application policy is possible.

This is an architectural argument under strong premises, not a formal proof of noninterference or a proof about an implementation. Its invariant concerns authorized plaintext holders, including holders of derivatives; it does not establish that an authorized reader uses information only for an intended purpose. Complete mediation, sound propagation of restrictions, private-key confinement, and correct policy enforcement require independent evidence. Cycles and retries preserve the argument only if every repeated transfer is mediated; cached authorization is subject to the continuity rule. No collection of passing quote-signature tests establishes these premises.

4. The proposed handoff contract

Each handoff should bind the information below into one authenticated transaction. This is a proposed integration contract, not a claim that a current AgenTrust schema implements these fields together.

Contract element	Required meaning
Transaction context	Protocol version, unique request identifier, challenge, expiry, sender and recipient identities, and session transcript binding.
Workload identity	Approved manifest and policy digests, accepted execution evidence, destination key, and verifier decision.

Contract element	Required meaning
Asset restrictions	Opaque asset identifier, confidentiality classification, permitted operation and audience, retention limit, and downstream delegation constraints.
Authority	Issuer-approved permission, parent delegation reference, attenuation constraints, and any applicable revocation state.
Platform requirements	Named policy version, accepted hardware class and security state, minimum security versions, debug restrictions, and required mitigations.
Output restrictions	Permitted return endpoint, inherited classification, approved declassification transform if any, and audit-disclosure policy.

The sender validates recipient evidence before sealing sensitive data. The recipient validates caller authority before opening or processing it. Where both sides release secrets or accept privileged actions, each appraises the other's execution environment. Mutual appraisal does not require claiming that both sides learn the outcome simultaneously; protocol sequencing and failure behavior must be specified.

Bind operation, audience, request identity, policy version, and channel context into authenticated data or a signed transcript. Otherwise, ciphertext and valid authorization may be transplanted into a different operation. A fresh challenge does not make an application request single-use: that requires request state or an idempotency policy covering retries, concurrent copies, and restarts.

Attestation follows the separation in the IETF RATS architecture: evidence is appraised against policy, and the relying party uses the result to decide access. An appraisal should carry its scope and expiry; a Boolean “verified” discards security-relevant distinctions. [RFC 9334](#)

Public-key sealing can use established constructions such as HPKE, with an appropriate authenticated protocol around them. Encryption alone supplies neither delegation authority nor proof of correct application behavior. This is a design recommendation, not a claim that cA2A's existing sealing format is HPKE-compatible. [RFC 9180](#)

5. Derivatives, declassification, and ordinary tools

A summary, embedding, classification, or generated tool argument may reveal its source. Changing representation does not establish a safe release. The conservative rule is to propagate the restrictions of all inputs that could influence an output. More precise release requires a justified transformation and the authority to approve it.

This follows established information-flow work: owners can impose dissemination rules, and declassification requires the relevant authority. The contribution proposed here is applying that discipline to appraised agent endpoints and explicit handoff evidence, rather than introducing a new information-flow theory. [Myers and Liskov, 1997](#)

Suppose the contract-analysis agent needs a public exchange rate. It can query with a currency pair and date if policy authorizes those values to leave. It must not include the contract paragraph merely because the tool call is permitted. A purpose string such as “financial analysis” does not constrain what the remote service does with plaintext.

There are three defensible outcomes at an ordinary external API:

- Send no protected information, after an approved transformation or selection establishes the permitted disclosure.
- Obtain authorization for a specified disclosure to that service, recording that the original operator-confidentiality boundary ends there for the released information.

- Refuse the operation.

A disclosure record should identify the authorized recipient, data category, transformation, approving authority, and resulting confidentiality scope. It must not claim that the recipient inherited a TEE guarantee. Once a recipient has learned a secret, revocation cannot make it forget. Revocation can stop future access; retention enforcement and key destruction can constrain future recovery only within their actual trust assumptions.

Prompt injection is relevant because it can cause an otherwise approved workload to propose an unauthorized disclosure. A trusted policy engine must evaluate actual destinations and payload restrictions independently of model instructions. Attestation establishes identity and reported state; it does not establish that a language model understands confidentiality correctly.

6. Release authorities, accelerators, and evidence

The release authority is part of the boundary. If an adversary can read model keys or replace the broker's accepted roots and policy, workload appraisal is bypassable. A protected broker needs owner-verifiable identity, protected provisioning, an authorized update path, and rollback resistance. Administrative separation within an organization does not constrain that organization if it can override both administrators. AWS Nitro Enclaves provides a concrete example: KMS key policies can condition operations on enclave measurements, and attested requests receive results encrypted to the public key in the attestation document. The deployment's policy administration remains a separate trust decision. [Nitro Enclaves with AWS KMS, Cryptographic attestation support in AWS KMS](#)

CPU and GPU evidence must describe the actual path. Two valid reports do not establish that the measured CPU workload used that GPU, that plaintext crossed a protected link, or that a fallback device was excluded. A common nonce can establish challenge freshness without proving co-location or a protected interconnect. A useful validation binds the transaction, device assignment, accepted modes, protected transport, and operation together. Multi-GPU systems must include their interconnects. NVIDIA documents distinct GPU and switch attestation requirements; device appraisal is one input to this broader validation. [NVIDIA Attestation SDK](#)

Evidence is another output channel. Tool arguments, response bodies, identity chains, and even unkeyed hashes of low-entropy secrets can disclose information. Prefer minimized records with opaque references, policy and decision identifiers, and protected evidence storage. Where equality checking is necessary, consider keyed commitments with explicit key ownership and rotation. Publishing a commitment may still reveal linkage or timing. A transparency log establishes publication properties within its design; it does not prove that omitted events never happened.

Time and revocation are bounded assumptions. Short leases constrain future key release but cannot erase keys already copied by an adversary. Wipe-on-expiry depends on trusted execution, memory handling, and a clock the adversary cannot stall. Verifiers should distinguish historical signature validity from present authorization and should record when revocation services could not answer.

7. Component experiments and their limits

The evidence below comes from separately pinned software, hosted Linux and cloud experiments conducted September 17 to 19, 2026. They are not one composed run. Test counts identify the scope of a particular execution; they do not measure confidentiality or add together into an assurance score. Synthetic attestation, emulated firmware, fresh hardware capture and owner-side replay remain separate evidence classes.

7.1 Response authentication

The opt-in cA2A profile ca2a-response-mac-v1 binds a response to the complete request occurrence and appraised responder key using X25519, HKDF and HMAC. A strict caller refuses missing, changed or unsupported proofs instead of accepting a legacy reply. The response commitment includes the request digest, status, kind and body. Per-occurrence state consumes verification once; expiry or a restarted verifier does not authorize replay. A submission timeout preserves an unknown execution outcome and does not cause automatic resubmission.

The implementation passed 57 focused software tests and 691 full-suite tests, with two full-suite skips, at the recorded revision. Its fixture cross-checks key derivation and authentication independently of the runtime helpers. This authenticates provenance or denial to the live caller. It supplies neither confidential output transport nor a portable signed execution receipt, and software-only peer evidence remains software assurance. The caller can compute the same MAC, so a third party cannot treat it as proof of who authored a response. [Response profile and implementation](#)

7.2 Confinement and a supervision counterexample

The native-Linux Docker reference places an adversarial agent behind a trusted host bridge and cMCP gateway. Its selected boundary includes a local Docker daemon, disabled container networking, restricted filesystem and process configuration, named tool adapters, and operator-owned sensitivity ceilings. Live policy restrictions serialize cutover with actual dispatch. Synthetic payloads probe network, tool, log, audit and telemetry boundaries. These controls address an agent escaping its configured release path; they do not hide plaintext from the host administrator or an approved tool.

Review exposed a supervision defect: a successful write and drain into a watchdog pipe does not prove the watcher is responsive. A suspended watcher could leave the container alive while the bridge continued to write. The repair requires an acknowledgment within one second, bounding write, drain and acknowledgment together. If the watcher is paused, the surviving bridge stops the container. The watcher separately stops it after loss of bridge heartbeats, subject to a responsive host and daemon.

The hosted Linux run passed 61 tests. The paused-watcher control terminated the container; a deliberately weakened write-only implementation left it running, causing the intended stop assertion to fail. Separate bridge kill/pause controls exercised the independent watcher. This causal result supports the tested supervision mechanism, not host-wide recovery, simultaneous loss of bridge and watcher, secure memory erasure or absence of every covert channel. [Confinement implementation](#), [Linux run](#)

7.3 Provisioning outcomes and continuity

WCM's owner and receiver bind provisioning to a fresh receiver key, appraised workload, effective configuration, model-key label and policy epoch. A receiver installs one authenticated envelope and refuses release before provisioning or after retirement. Owner epoch storage constrains older policy only while its database and namespace remain authoritative. Storage deletion, substitution, snapshot rollback and separate clones can restore acceptance of an older policy. Tests make this trust assumption observable rather than describing SQLite persistence as rollback resistance.

The lifecycle experiment added 18 software cases to 118 existing focused controls: 136 passed; the full local run passed 995 tests with 32 skips. Real loopback HTTP exchanges used synthetic signed SNP reports. Lost acknowledgments were induced before and after installation; a false acknowledgment showed that an HTTP receipt does not establish installation. Four deliberate weakenings made their tests fail: nonce consumption, install expiry, installation-proof disclosure and teardown after clock failure.

A worker control opened a real software release envelope and attached a fake loaded worker through the existing serving-stop adapter. It exercised the declared callbacks and simulated clocks. It did not execute a model in protected hardware or demonstrate real suspend/resume, trusted time, accelerator cleanup or erasure. Retirement and epoch advancement cannot recall an already delivered key. [Lifecycle contract and tests](#)

7.4 Measured-image candidates and cloud counterexamples

A restricted OVMF candidate and boot loader constrain selected launch inputs. Eleven complete-image TCG cases included a valid boot, malformed or missing launch tables, substituted kernel/initrd/command line/header, a named denied image and a weakened hash-comparison control. The test variants inject synthetic launch state and include diagnostic instrumentation. Passing these controls does not demonstrate that the exact production image enforced the same property on SNP hardware. Actual built-artifact prediction and seven digest-sensitivity controls are also software results.

A separate Azure SNP/vTPM experiment on September 18 authenticated fresh evidence; seven controls passed on both guest and owner. After a nonsecret probe file changed, the provider still quoted the old caller-supplied digest and the verifier accepted that digest. This counterexample demonstrates that authenticated caller-selected data does not establish measurement or enforcement of the current application. It is not a bypass of the restricted firmware, which was not loaded.

On GCP, owner-side verification accepted two fresh native SNP reports and rejected five altered-evidence or trust-input cases. A third native provisioning report carried the requested binding. Changing a nonsecret application probe between reports left the launch measurement unchanged. The native collector therefore worked, but neither the unchanged measurement nor the caller binding established the identity or custody of the subsequently installed broker code. Managed guests lacked the launch-host access needed to run the custom firmware. Both clouds' temporary resources were deleted.

The missing experiment is a launch under owner-approved firmware, kernel, application and configuration, with a predicted measurement chosen before collecting a fresh native report. The controlled-host prerequisite remains open. No stronger application-identity claim is inferred from authentic cloud reports. [Firmware and hardware diagnostic work](#)

7.5 Exact-output disclosure

The cMCP disclosure adapter requires independently scoped owner approval for an external or less restricted release. An Ed25519 approval binds exact output bytes through their digest, inherited labels, source scope, workload, recipient, purpose, policy version, nonce and validity. Changing the bytes or those bindings requires a new approval. Model-generated labels cannot replace accumulated restrictions. The adapter is an opt-in library path, separate from gateway sink ceilings; it is not a gateway bypass parameter or a semantic classifier.

A trusted recipient registry selects delivery callbacks. A durable transaction consumes the request identifier before callback execution. An exception or lost acknowledgment leaves delivery unknown and the identifier consumed; a normal return is an acknowledgment, not proof of processing or future confidentiality. Classification, source completeness, transport identity, clocks and replay storage remain trusted. Deleting or rolling back the store defeats its replay guarantee. Approval digests stay private because low-entropy outputs can be guessed.

The focused run passed 77 tests, the full run 2,046 with 20 skips. Six deliberate gate weakenings each caused an assertion failure. These results support the sampled exact-output checks. They do not establish integration with confinement or automatically justify release of summaries and embeddings. [Disclosure contract and implementation](#)

7.6 Historical live handoff diagnostic

On September 17, two temporary GCP AMD Milan SNP guests completed paced cA2A handoffs in both directions: one positive and ten expected rejections per direction, 22 cases total. Receiver-side logs observed hardware caller appraisal and expected nonsecret payload digests. Both guests belonged to one operator and one project. Measurement pins came from authenticated reports, not an independently predicted application image. The reported platform state lacked ciphertext hiding and enabled SMT; the diagnostic explicitly accepted that narrower profile. Stronger policy settings rejected those hosts.

The executed snapshot used unauthenticated HTTP responses and unsigned receipts. The later response-MAC software tests do not upgrade this historical run. An earlier burst timed out in each direction; the cause remains unisolated. The paced run supports live mutual appraisal and selected sealed-delivery rejection paths. It does not establish independent-operator composition, application confinement, protected model execution or a stronger hardware posture. Both guests and disks were deleted. [Diagnostic and response follow-up](#)

8. Composed software experiment and remaining acceptance

The pinned development harness executes provisioning, diagnostic model computation, a mediated tool, delegated peer, response verification and exact-output disclosure in one transaction. A native Linux profile inserts a confined adversarial agent at the tool boundary. The recorded hosted run passed 36 tests in 19.24 seconds. This is a synthetic-attestation, same-operator software result. The harness merged into WCM main with pull request 148 on September 20, 2026, and its standalone reproduction bundle with pull request 151 on September 22.

[Implementation, reproduction bundle, recorded hosted run](#)

For this edition the harness was run again at WCM main 94d5519, with the same cMCP, cA2A and confinement revisions. The hosted workflow on that commit passed 36 tests in 18.76 seconds, then rebuilt the bundle, fetched every source afresh into a new workspace and passed 36 tests in 18.60 seconds. A fresh-environment bundle run on Windows, where the native profile is unavailable, passed the 32 portable tests in 18.56 seconds. Ignoring transaction identifiers and the rebuilt container image digest, every observation record matched the recorded run. Between the two WCM revisions, one mutation helper changed its signature to follow an unrelated key-broker interface change. [Rerun on WCM main](#)

8.1 Executed path and observations

Test-owned roots sign synthetic SNP evidence. WCM owner provisioning, receiver installation, workload appraisal and sealed key release precede AES-GCM decryption of a nonsecret affine model. A fresh transaction identifier appears inside the encrypted model and its authenticated associated data. The plaintext model digest enters the owner-pinned manifest and provisioning policy. The host computes $3 * 7 + 2$ and carries the transaction and value 23 forward.

A cMCP/Cedar gateway dispatches a JSON-RPC subprocess tool. The actual tool return becomes the sealed cA2A payload to a loopback HTTP peer, whose local policy restricts delegation scope. The caller checks the response MAC and transaction record identifier before a scoped owner approval authorizes exact bytes for the final registered recipient. Controller checks bind these stages; they are trusted application code, not a new interoperable transcript schema. The model manifest is owner-pinned; this fixture does not test a model publisher's signature.

The native profile gives the already computed bytes to an adversarial Docker agent. It attempts IPv4 TCP, IPv6 TCP, UDP, a filesystem export and stderr disclosure, then requests one permitted and one forbidden tool. The bridge routes the permitted call through the same gateway and stops/removes the agent before the tool result

proceeds to the peer and disclosure gate. Tool-side subprocess records, peer-side decrypted bytes and final callback bytes must each equal the expected transaction and value. These are distinct observation points under one test operator, with shared fixtures and controller assumptions.

8.2 Paired controls and causal results

Test group	Cases	Required observation
Portable valid path	1	One authorized delivery; reopening replay storage still refuses reuse.
Portable refusal paths	13	Selected key, evidence, model, tool, delegation, response or disclosure gate prevents final disclosure.
Ambiguous callback outcome	2	Failure before or after actual receipt remains unknown; request stays consumed without retry.
Portable gate mutations	16	Removing the targeted rule changes the observation and makes its intended oracle fail.
Native valid path	1	Authorized delivery succeeds; selected network, filesystem and persistent-log probes observe no leak.
Native isolation mutations	3	Weakened networking, filesystem or logging produces the corresponding observed leak.

The portable refusal set covers substituted workload, channel key, manifest configuration, encrypted model and transaction; missing evidence or approval; forbidden tool, excess delegation scope and software downgrade; changed response MAC, revoked disclosure authority and changed approved bytes. Sixteen mutations cover these thirteen refusals, replay and both unknown-outcome cases. The runner requires equality between the named scenario set and mutation-target set. This establishes causal coverage of that enumerated set, not every protocol rule, input shape, concurrency schedule or downgrade mechanism.

Each mutation must first produce its predicted unsafe delivery, replay or falsely acknowledged outcome; the test then requires the corresponding acceptance assertion to fail. For example, the channel-key mutation permits an attacker-owned key to open the release when quote binding is removed. The model mutation removes AES-GCM tag finalization while retaining real decryption, making changed-tag plaintext usable. These controls test consequential behavior rather than merely changing a return label.

In the native valid case, all three network observers recorded zero matching receipts; filesystem and persistent-log probes found no payload; the forbidden public tool was not invoked; and the agent was stopped. One allowed and one denied gateway operation were observed. Each native mutation was first rejected by normal container-profile inspection. The test then bypassed that inspection to establish causality: host networking yielded one matching receipt at each network observer, a writable filesystem exposed the export, and persistent logging exposed stderr.

All three weakened native cases still completed authorized final delivery. A successful final receipt therefore cannot establish that earlier plaintext exits were controlled. The isolation assertions detect those earlier leaks separately. This result is specific to the probes and configuration tested; it does not establish absence of other exits or covert channels.

8.3 What this run does not establish

The host, kernel, Docker daemon, bridge, broker, model computation, tool and peer remain trusted ordinary processes. The model executes on the host before the agent receives its result. No hardware protects the model key or computation from that operator. Synthetic SNP signatures exercise verification paths against test-owned roots; they do not attest a live workload. The native suite does not repeat every supervisor failure from the separate component experiment. Teardown proves observed process termination, not secure erasure.

Provisioning and key-release stage labels describe local API observations. They do not establish remote installation under lost acknowledgment; the separate lifecycle experiment addresses that ambiguity. The harness does not provide independent operators, a trusted clock, rollback-proof replay storage, production LLM execution, a protected accelerator path or semantic justification for declassification. No claim about the complete confidentiality boundary follows from the test count.

On September 19, downloaded PyPI wheels `cmcp-runtime==0.5.0` and `ca2a-runtime==0.2.0` lacked the disclosure and response modules. The later releases `cmcp-runtime==0.6.0` (September 25) and `ca2a-runtime==0.3.1` (September 26) ship `disclosure.py` and `response.py` byte-identical to the pinned source revisions. This is a wheel inspection; the harness has not been run against the released packages. The confinement adapter remains in cMCP's example sources and is not in the wheel, and the harness still imports WCM test fixtures. A released-package integration therefore still needs those dependencies packaged or replaced, then a package-only run of the acceptance controls. [Integration acceptance issue](#), [contribution rules](#)

8.4 Remaining hardware execution contract

Status on September 27, 2026: not started. It waits on a controlled SNP launch host (WCM issue 149) or partner-operated execution; identity, lifecycle and protected execution acceptance (WCM issues 144, 145 and 146) remain open. [Controlled-host prerequisite](#), [acceptance backlog](#)

Begin with an owner-approved, nonsecret deterministic CPU model and one input. This is a diagnostic fixture for the custody path; it is not evidence about production model accuracy or GPU protection. Pin the artifact, runtime, effective policy, launch inputs and channel keys before admission. Use independently supplied trust roots and reference values. A production confidentiality claim additionally requires protected broker key generation, model decryption and loading inside the accepted measured boundary.

Record distinct observations for authorization, envelope dispatch, receipt, installation, decryption, load, inference, response authentication, disclosure and teardown. Each observation identifies its producer, authenticated transaction binding, revision and evidence class. Report each outcome as established, contradicted or unavailable. Missing instrumentation is unavailable. A timeout after dispatch leaves the remote outcome unknown; it does not imply refusal or justify another execution.

Observe ciphertext/key release from the owner side and load/inference from the accepted runtime, with an independent receiver or verifier at each external boundary. A trusted runtime receipt can establish only what its measured implementation actually observes. It cannot independently prove erasure, absence of hidden copies or semantic correctness. Public artifacts use opaque references; raw reports, private payloads and guessable confidential digests remain in controlled storage.

Enumerate plaintext in guest buffers, temporary files, caches, model staging, swap, crash services, logs and outputs. The CPU-only run must declare no accelerator path. A later GPU experiment must bind device identity and accepted mode to the actual protected CPU/device channel and executing model, reject unrelated valid device evidence, and account for multi-device links. A GPU verifier result alone does not satisfy that test.

8.5 Remaining deployment acceptance matrix

The matrix below specifies the broader deployment target. Section 8.2 covers selected software cases within it; the remaining platform, concurrency, runtime-change and lifecycle cases need deployment evidence. Every row needs a successful valid control, observation at the operation boundary, and a deliberate weakening that makes the intended rejection assertion fail. If an earlier parser or schema blocks a case, that case does not demonstrate the later gate. Mutations and acceptance oracles must be recorded separately.

Change under test	Boundary observation required	Status, September 27, 2026
Substitute workload or effective configuration	Owner withholds the key before provisioning to the substituted image.	Software control and mutation passed (synthetic attestation). Controlled hardware: not started.
Substitute recipient or channel key	No protected bytes reach the substituted endpoint.	Software control and mutation passed. Controlled hardware: not started.
Downgrade software or platform mode	Admission refuses before key release or payload sealing.	Software downgrade passed. Platform mode: GCP diagnostic hosts were rejected under stronger policy; no controlled-host test.
Substitute model or runtime after appraisal	Decryption/load refuses the substituted artifact before execution.	Changed encrypted model refused in software. Runtime substitution after appraisal: not tested.
Replay across concurrency or restart	Authoritative state rejects reuse; snapshot assumptions remain explicit.	Sequential replay after reopening storage refused in software. Concurrency and rollback-proof storage: not tested.
Omit required evidence	No fallback promotes missing proof to established assurance.	Software control and mutation passed. Controlled hardware: not started.
Attempt direct egress or forbidden disclosure	Independent sink observes no protected delivery; valid authorized delivery succeeds.	Native Linux profile passed with same-operator sinks. Independently operated sinks: not started.
Revoke after approval and before use	Future admission stops under the stated cutover rule; in-flight outcomes remain separate.	Revoked disclosure authority refused in software. Deployment cutover: not tested.
Lose acknowledgment after dispatch	Sender reports unknown; no blind retry; independent receiver distinguishes the two actual outcomes.	Software callback and loopback HTTP cases passed. Independent receiver: not started.
Fail clock, supervisor or teardown	Declared admission/stop behavior is observed; inability to establish cleanup remains visible.	Supervisor pause and simulated clock failure passed in software. Trusted clock and hardware teardown: not started.

8.6 Reproducibility and closure

A run manifest must contain the selected claim, asset and principal roles, permitted recipients, trusted components, policy/trust-input references, transaction-binding format, exact source and package digests, commands and observation producers. Each result retains its own evidence class: synthetic, emulated, replayed hardware or fresh hardware. Fresh hardware in one component never promotes another component's synthetic result. Sanitization must preserve verifiability or state what the public reader cannot reproduce.

The development harness retains source revisions, harness hashes, installed dependency versions, commands, policy and trust descriptions, and 36 minimized observation records. Its environment is recorded per run, not completely hash-locked. The published software milestone additionally requires installable fixture and

confinement-adapter support and repetition of the acceptance controls without source-tree imports; the disclosure and response APIs are now released. The live milestone additionally needs approved measured application identity, validated hardware lifecycle and independently operated peers. WCM source is publicly accessible as of September 20, 2026. Portions of the hardware evidence remain access-controlled; the source-based software bundle does not reproduce those private observations. The parent research claim remains open until one selected deployment satisfies the composed criteria. The live milestone requires hardware access; the published software milestone requires packaging of the remaining development-only dependencies.

9. Findings and open questions

The experiments identify three specific ways an apparently successful check can overstate a confidentiality boundary. A fresh authenticated quote can carry a stale caller-chosen application digest. A successful heartbeat write can coexist with an unresponsive supervisor. A successful or missing HTTP acknowledgment can fail to establish key installation. Exact-output approval and request-bound response authentication address additional software boundaries, but neither proves the complete workflow.

The conditional composition argument still depends on measured application identity, key confinement, complete mediation and continuity at every plaintext holder. Those premises have not been established together. The composed software result establishes selected admission and egress controls within a trusted-host workflow. The next empirical target is moving key handling and model execution into an owner-appraised controlled launch, followed by independent-peer reproduction. The semantic research question remains how to justify narrower releases of model-derived information without treating the model's own judgment as authorization.

Appendix A. Revision and reproduction ledger

The following are evidence snapshots, not interchangeable descriptions of a latest branch. Review and merge state is maintained in the project tracker rather than used as an assurance level. WCM repository links are publicly accessible as of September 20, 2026; hardware evidence access remains separate.

Result	Source snapshot	Reproduction entry point
Response authentication	cA2A fc22c6448461	tests/unit/test_response_authentication.py; response-MAC fixture and profile.
Supervision repair	cMCP bad751becca0	Confinement workflow run 35445862949; 61 native Linux tests.
Provisioning lifecycle	WCM db8f10638bb9	python/tests/test_provisioning_lifecycle.py; full suite 995 passed, 32 skipped.
Firmware prediction controls	WCM 050002114123	python/boot/prediction_controls.py; retained built-image and VM artifacts.
Azure hardware diagnostic	WCM 1f5092256f7f	September 18 capture and owner replay; diagnostic recorded separately in af5468e70964.
GCP native diagnostic	WCM af5468e70964	September 18 native reports and owner replay; receipt update dc5dd52.
Exact-output disclosure	cMCP 2cdb168ce520	tests/unit/test_disclosure.py; disclosure contract and mutation records.
Composed software and confined agent	WCM PR head f168aa861c0e; tested merge 15e67c2fb75b	Workflow run 35469947313; python/composed/run.py; 36 tests.

Result	Source snapshot	Reproduction entry point
Composed rerun for this edition	WCM main 94d5519fd604	Workflow run 36272971255; python/composed/reproduce.py; 36 tests hosted, 32 portable on Windows.

The composed run used cMCP 2cdb168ce520, cA2A fc22c6448461 and confinement adapter bad751becca0. Its manifest records full revisions and the built container image identity. The WCM test-merge revision 15e67c2fb75b differs from the reviewed PR head because GitHub executed its generated merge checkout. The September 27 rerun used the same cMCP, cA2A and confinement revisions.

The canonical implementation record retains full revisions, commands, outputs, private-evidence locations and cleanup receipts. The paper reports selected bounded results. It does not redistribute device-identifying hardware bundles, private keys or protected data. A reviewer without the private evidence can assess the public software tests and the stated limits, but cannot independently reproduce every reported hardware observation from this PDF alone.